

Backend PRD

FOR THE BACKEND LEAD

SureJob | POC for the InnovateX 2026 Regional Stage | Track A: Inclusive Finance

Yaba College of Technology | Friday 9 October 2026 | Prepared Tuesday 6 October

Own the job lifecycle and the three integrations.

Owner	_____ (Backend lead)
Depends on	Integrations owner (adapter implementations, sandbox keys). Frontend (signs off the API contract tonight).
Hands off to	Frontend: live API by Thu noon. Pitch lead: a stable demo mode and a reset endpoint.
Deadlines	Wed night: happy path on mock adapters. Thu noon: all adapters work in both modes. Thu 6pm: freeze.

1 Purpose and outcome

Build the Go service that owns the job lifecycle. It exposes a small REST API to the frontend and talks to Ecobank and Curacel only through adapters. Everything runs offline on mocks, and each integration flips to its live sandbox with one environment variable.

Success: a single script books a job, gets a quote, pays into escrow, confirms from both sides and prints a payout code. A second run files a claim. No manual database edits.

2 Scope

IN: we build this	OUT: not before Friday
✓ Job state machine with enforced transitions	✗ Login, KYC, roles and permissions
✓ Eight REST endpoints (section 5)	✗ Webhooks and background jobs (adapters are synchronous)
✓ SQLite storage, seed data, reset endpoint	✗ Real money and production keys
✓ Three adapter interfaces, each with a mock and a live version	✗ Cloud deployment (it runs on a laptop for the pitch)
✓ LIVE or MOCK per adapter, reported back to the UI	✗ Rate limits, metrics, multi-tenancy
✓ Event log per job (feeds the timeline), CORS for Next.js, demo.sh	✗ Retries and queues beyond the pay retry in section 8

3 Stack and layout

Go 1.22 or newer, standard library **net/http** (or chi). SQLite through a pure-Go driver such as **modernc.org/sqlite**, which avoids cgo build problems on the demo laptop. One binary, config from env, money stored as integer kobo.

cmd/api/main.go	wiring, env, server start
internal/httpapi/	handlers, JSON helpers, error format, CORS
internal/jobs/	state machine and service logic
internal/store/	SQLite queries, migrations, seed
internal/adapters/	payment/ insurance/ payout/ (interface + mock + live)
scripts/demo.sh	curl walkthrough of the happy path

4 Job state machine

From	To	Trigger	Side effects
BOOKED	BOOKED	POST /quote	Insurance.Quote. Store quote id and premium. No state change.
BOOKED	ESCROWED	POST /pay	Payment.Collect(job amount + premium). Store escrow ref.
ESCROWED	INSURED	Same request, after Collect	Insurance.Issue. Store policy ref.
INSURED	CONFIRMED	Second party confirms	Store both confirmation times.
CONFIRMED	PAID_OUT	Immediately after	Payout.IssueToken. Store code, ref, expiry.
INSURED or CONFIRMED	CLAIM_FILED	POST /claim	Insurance.FileClaim. Payout is held.

Anything else returns 409 with code INVALID_TRANSITION. Each transition runs in one database transaction and writes an event row. CONFIRMED is transient because payout fires right away, so in practice claims come from INSURED.

5 API contract (frontend signs this off tonight)

Endpoint	Request body	Returns	Notes
GET /api/config	none	{ modes }	Which adapters are mock or live right now.
POST /api/demo/reset	none	{ jobId }	Reseeds users and the demo job.
POST /api/jobs	customerId, workerId, title, amountKobo	Job (BOOKED)	Validates amount.
GET /api/jobs/:id	none	Job	Includes events and modes.
POST /api/jobs/:id/quote	none	quoteId, premiumKobo, totalKobo, coverage[]	Coverage is a list of plain strings.
POST /api/jobs/:id/pay	quoteId	Job (INSURED)	Collect then Issue. Safe to retry.
POST /api/jobs/:id/confirm	party: customer or worker	Job	PAID_OUT after the second confirm.
POST /api/jobs/:id/claim	filedBy, reason, details	Job with claim	Reasons: damage, injury, not_done.

```
Job = { id, title, status, amountKobo, premiumKobo, totalKobo,
  customer:{id,name}, worker:{id,name},
  confirmations:{customer:bool, worker:bool},
  escrowRef, policyRef, claim:{ref,status,reason} | null,
  payout:{code,expiresAt,ref} | null,
  modes:{payment,insurance,payout}, // "mock" or "live", as actually used
  events:[{type,at,detail}] }
Errors: 4xx or 5xx with { error:{ code, message } }
```

No auth in the POC. The UI only shows payout.code on the worker view. Do not describe this as secure in the pitch.

6 Data model

Table	Fields
users	id, name, role (customer or worker), phone
jobs	id, customer_id, worker_id, title, amount_kobo, premium_kobo, status, quote_id, escrow_ref, policy_ref, payout_code, payout_ref, payout_expires_at, customer_confirmed_at, worker_confirmed_at, modes_json, created_at
claims	id, job_id, filed_by, reason, details, ref, status, created_at
events	id, job_id, type, detail, created_at

Seed: customer Tunde, worker Emeka (mechanic), job "Brake repair" at 1,500,000 kobo (₦15,000). POST /demo/reset restores exactly this.

7 Adapters

Adapter	Method	Live target	Mock behavior
Payment	Collect(jobID, amountKobo)	Ecobank Local Bank Payment API, sandbox	Ref ESC-xxxx after about 500ms.
Insurance	Quote(value, duration, category)	Curacel Grow: Quotations	Premium is 2% of job value, minimum ₦200, rounded up to ₦50 (illustrative). Returns quoteld and coverage list.
Insurance	Issue(quoteld, jobID)	Curacel Grow: Orders, then Policies	Policy ref POL-xxxx.
Insurance	FileClaim(policyRef, reason)	Curacel Grow: Claims	Claim ref CLM-xxxx, status received.
Payout	IssueToken(jobID, amountKobo, phone)	Ecobank XpressCash Token Service	8-digit code, 24h expiry. Placeholder shape until the sandbox responds.

Ownership: the backend lead defines the three interfaces tonight. The integrations owner writes the mocks and the live versions against each provider's docs. A mock must return the same shape as the live call, so the swap is config only.

Mode switch: env PAYMENT_MODE, INSURANCE_MODE, PAYOUT_MODE, each **mock** (default) or **live**. Keys come from env only and are never committed.

No silent fallback. If a live call fails, return the error and let the operator flip to mock. Quietly falling back would present a simulated result as live. The mode actually used is stored on the job and returned in **modes**, which the UI shows as a Sandbox or Simulated badge.

8 Rules and edge cases

Area	Rule
Idempotency	Repeating /pay on an ESCROWED job only retries the policy step and never charges twice. The same party confirming twice is a no-op. Payout is issued once (guard on payout_ref).
Validation	Amount above zero and below a sane cap (suggest ₦500,000). Unknown job returns 404. Bad party value returns 400.
Concurrency	SQLite with one writer. Wrap each transition in a transaction.
Latency	Mocks add 300 to 800ms so the demo feels real. Make it an env value so rehearsals can run fast.
Failure drill	Should have: MOCK_FAIL=quote, pay or payout makes the next call fail, so the team can rehearse recovery.
Logging	One line per transition and per adapter call, with the mode.

9 Acceptance checklist

- ☐ scripts/demo.sh runs book, quote, pay, confirm twice, and prints a payout code
- ☐ Claim from INSURED returns a claim ref and sets CLAIM_FILED
- ☐ Confirming twice never issues a second payout
- ☐ Changing an env var changes /api/config and the modes on new jobs
- ☐ POST /api/demo/reset restores the seed job in under a second
- ☐ Cold start and a full run work with no internet

10 Timeline

When	Target
Tue night	Interfaces and API contract frozen with Frontend and Integrations. Repo skeleton, schema, seed.
Wed	State machine and all endpoints on mock adapters. demo.sh passing by night.
Thu morning	Wire live adapters where keys exist. Flip tests. Fix contract bugs the frontend finds.
Thu 6pm	Freeze and tag the demo build. Bug fixes only after this.
Fri	Start from the tagged build on the demo laptop and run demo.sh as the smoke test.